SDEV 2451

Full Stack Web Development

Frontend and Backend API Integration - 3

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments

- No Cheating

- Be a good classmate

- Don’t waste your time

- Show up to class

Agenda

On the right is what we will cover today.

- The Mockup → Code Workflow
- Analyzing a Mockup for Components
- Installing and Setting Up react-leaflet
- MapContainer and TileLayer
- Drawing a Route with Polyline
- Auto-fitting Bounds with useMap
- Status Mutations with useMutation
- Key Takeaways

The Mockup → Code Workflow

Before writing any code, start with a UI mockup. It forces you to think through layout, data requirements, and interactions — without getting distracted by implementation.

The four-step process for building a full-stack feature from a mockup:

1. **Design the mockup first** — sketch the UI before touching a file. Work with a design team, or create it yourself using a tool like Excalidraw or Figma.
2. **Analyze the mockup** — identify which components already exist, which need to be built, and which the component library covers.
3. **Build with mock data first** — implement every component against static fixture data so you can verify the layout and logic before the backend is involved.
4. **Connect to the real API** — swap the mock data import for a React Query hook. The components don't change — only the data source does.

The Mockup → Code Workflow (continued)

- Starting from a mockup separates concerns: you think about design before worrying about network requests, and you think about API contracts before writing fetch logic.
- Steps 1–3 can be done entirely without a running backend, which lets frontend and backend work proceed in parallel.
- The mock-first → real-data transition is mechanical — it confirms the component API (props) is correct before any live data flows through it.
- This workflow applies to any full-stack feature, not just detail pages: dashboards, forms, and list views all benefit from the same four-step discipline.

Analyzing a Mockup for Components

Let’s go analyze the mockup in the example that was provided.

Installing and Setting Up react-leaflet

`react-leaflet` wraps the Leaflet mapping library in React components. Install it alongside the already-present `leaflet` package.

```
npm install react-leaflet
```

Then import the Leaflet CSS anywhere it will be bundled — typically inside the component that renders the map:

```
// src/components/LocationMap.jsx

import { MapContainer, TileLayer } from 'react-leaflet'
import 'leaflet/dist/leaflet.css'

function LocationMap() {
  return (
    <MapContainer center={[51.505, -0.09]} zoom={13}
      style={{ height: '300px', width: '100%' }}>
      <TileLayer url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
        attribution="&copy; OpenStreetMap contributors" />
    </MapContainer>
  )
}
```

Installing and Setting Up react-leaflet (continued)

- `leaflet` (the core JS library) and `react-leaflet` (the React wrapper) are separate packages. You need both — `react-leaflet` does not bundle Leaflet.
- `import 'leaflet/dist/leaflet.css'` is **mandatory**. Without it, map controls and tiles render in the wrong positions even though the JS works fine.
- `MapContainer` must receive either `center` + `zoom` or `bounds` as initial props — it will throw without them.
- `style={{ height: '300px' }}` is required. Leaflet renders into a fixed-size container; without an explicit height the map collapses to zero pixels and appears invisible.
- `scrollWheelZoom={false}` is a useful default — it prevents the map from hijacking the browser's scroll when the user is scrolling the page.

Drawing a Route with Polyline

`Polyline` draws a line between an ordered list of lat/lng positions. Combine it with `pathOptions` to style the line.

```
import { MapContainer, TileLayer, Polyline } from 'react-leaflet'
import 'leaflet/dist/leaflet.css'

function RouteMap({ startCoords, endCoords }) {
  const positions = [
    [startCoords.lat, startCoords.lng],
    [endCoords.lat,   endCoords.lng],
  ]

  return (
    <MapContainer center={positions[0]} zoom={10}
      style={{ height: '300px', width: '100%' }}>
      <TileLayer url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
        attribution="&copy; OpenStreetMap contributors" />
      <Polyline positions={positions}
        pathOptions={{ color: '#570df8', dashArray: '10, 8', weight: 3 }} />
    </MapContainer>
  )
}
```

Drawing a Route with Polyline (continued)

- `positions` is an array of `[lat, lng]` tuples. Two points draw a straight line; add more points to trace a complex path.
- `pathOptions` is a plain object passed to Leaflet's path layer — any Leaflet `Path` option is valid here.
- `color` accepts any CSS colour value; `dashArray: '10, 8'` produces a dashed line (10 px dash, 8 px gap) that visually distinguishes the route from solid road lines on the base map.
- `weight: 3` sets line thickness in pixels. Thicker lines (4–5) work well for routes on zoomed-out maps.
- Guard the component with `if (!startCoords || !endCoords) return <placeholder />` — Leaflet will throw if you pass `null` values into `positions`.

Auto-fitting Bounds with useMap

`useMap()` returns the Leaflet map instance — but it can only be called from inside a component that is a *child* of `MapContainer`.

```
import { useEffect } from 'react'
import { MapContainer, TileLayer, Polyline, useMap } from 'react-leaflet'

function FitBounds({ positions }) {
  const map = useMap()
  useEffect(() => {
    map.fitBounds(positions, { padding: [50, 50] })
  }, [map, positions])
  return null
}

function RouteMap({ startCoords, endCoords }) {
  const positions = [[startCoords.lat, startCoords.lng],
                     [endCoords.lat,   endCoords.lng]]
  return (
    <MapContainer center={positions[0]} zoom={7} style={{ height: '300px', width: '100%' }}>
      <TileLayer url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
        attribution="&copy; OpenStreetMap contributors" />
      <Polyline positions={positions}
        pathOptions={{ color: '#570df8', dashArray: '10, 8', weight: 3 }} />
      <FitBounds positions={positions} />
```

Auto-fitting Bounds with useMap (continued)

- `useMap()` is a react-leaflet hook that returns the underlying Leaflet `map` object. It only works inside a descendant of `MapContainer` because `MapContainer` provides the map via React context.
- `FitBounds` is a **behaviour-only component** — it returns `null` and has no visual output. Its only job is to call `map.fitBounds()` after the map has mounted.
- `useEffect` is required because `fitBounds` must run *after* the map has initialised. Calling it during render would attempt to interact with a map that doesn't exist yet.
- `{ padding: [50, 50] }` adds 50 px of breathing room on each side so neither endpoint is clipped at the viewport edge.
- `center` and `zoom` are still required on `MapContainer` for its initial render; `FitBounds` overrides them immediately after mount so their exact values don't matter.

Status Mutations with useMutation

`useMutation` is React Query's tool for write operations — the equivalent of `useQuery` but for POST/PATCH/DELETE calls.

```
// src/hooks/useEventDetails.js

import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { fetchEventDetails, fetchStartEvent, fetchCompleteEvent } from '../api/events'

export function useEventDetails(id) {
  const queryClient = useQueryClient()

  const { data: event, isLoading } = useQuery({
    queryKey: ['event-details', id],
    queryFn: () => fetchEventDetails(id),
    enabled: !!id,
  })

  const startEvent = useMutation({
    mutationFn: () => fetchStartEvent(id),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['event-details', id] }),
  })
```

Status Mutations with useMutation (continued)

- `useMutation` returns an object with `.mutate()` to trigger the call and `.isPending` to track whether a request is in flight — use `isPending` to disable buttons and show a spinner.
- `onSuccess: () ⇒ queryClient.invalidateQueries( ... )` marks the cached entry as stale after a successful write. React Query re-fetches in the background so the UI reflects the new server state without a page reload.
- Returning the full mutation object (not just `.mutate`) lets the component access `.isPending` — useful for disabling a button while the request is in flight and preventing double-submissions.
- Conditional button rendering based on `event.status` ensures the UI always shows only valid next actions — a completed event shows nothing, a pending event shows "Start", an in-progress event shows "Complete".

Challenge — Add Markers at Route Endpoints

The route currently shows a dashed line but no markers at either end.

1. Add a `Marker` component from `react-leaflet` at both the start and end `positions`.
2. Fix the default Leaflet icon path issue that occurs with Vite bundling (search "leaflet marker icon vite" for the standard fix).
3. Add a `Tooltip` to each marker that shows the location name (`startLocation` / `endLocation`) when the user hovers over it.

Key Takeaways

- **Design the mockup before writing code** — a mockup separates design thinking from implementation and gives you a concrete task list before any files are opened
- **Colour-code the mockup** — green (already built), purple (build from scratch), orange (component library) makes the work list explicit
- **Guard before rendering Leaflet** — always check `!startCoords || !endCoords` before rendering `MapContainer`; Leaflet throws on null values
- `import 'leaflet/dist/leaflet.css'` **is mandatory** — omitting it breaks map layout even though the JS still runs
- `useMap` **requires a child component** — `useMap()` only works inside a descendant of `MapContainer`; `FitBounds` is the standard pattern for any code that needs the Leaflet map instance

Example

Let’s go do an example together